Lab 2B: Hooks and Human-Invoked Skills

Duration: 30 min

After: Hooks & Human-Invoked Skills lecture (Day 2, Block 2)

Prerequisites: Lab 2A complete (code-review skill)

Objective

Choose from the current **31 hook events**, wire a deterministic PostToolUse command hook, build a secret blocker in two stages - **advisory (exit 0)** then **blocking (exit 2)** - watch Claude react to the block feedback, scope it with a **matcher**, and build a parameterized **human-invoked skill** using `$ARGUMENTS`.

Deliverable (toolkit chain 6 of 12)

Toolkit artifacts #3 and #4: registered hooks (`.claude/hooks/` + `.claude/settings.json`) and a human-invoked review skill (`.claude/skills/review/SKILL.md`).

> ***START MARKER:*** *In* `business-dashboard`*,* `/clear` *done.*

Fell behind? Run:

```
Set up a code-review skill at .claude/skills/code-review/SKILL.md with a precise trigger
description, disable-model-invocation: false, read-only allowed tools, and RED/YELLOW/BLUE
review instructions. Also create CLAUDE.md with basic project rules.
```

Exercise 0: Choose the Event and Handler (3 min)

Hooks are external handlers that run on lifecycle events. The current reference contains **31 events**; learn the high-value pairs and use the Hooks Events Reference for the full map. There are **5 handler types**: `command`, `http`, `mcp_tool`, `prompt`, and `agent`. Today you build deterministic `command` handlers.

Group	High-value events
Tools and permissions	**PreToolUse**, **PostToolUse**, PostToolUseFailure, PermissionRequest, PermissionDenied
Session and prompt	SessionStart, SessionEnd, UserPromptSubmit, Stop, Notification
Agents and workspace	SubagentStart, SubagentStop, PreCompact, PostCompact, WorktreeCreate

Warm-up: format after an edit -> **PostToolUse**. Stop a secret before it lands -> **PreToolUse**.

Exercise 1: PostToolUse Format Hook (7 min)

Create the formatting hook (advisory by nature - it fixes, never blocks):

```
Create .claude/hooks/post-edit-format.sh that:
- Reads JSON from stdin
- Extracts .tool_input.file_path
- Exits 0 when there is no path
- Processes only .js/.ts/.jsx/.tsx files
- Runs npx prettier --write when npx is available
- Is executable

Merge a PostToolUse hook into .claude/settings.json:
{
"hooks": {
"PostToolUse": [
{ "matcher": "Write|Edit",
"hooks": [
{ "type": "command", "command": ".claude/hooks/post-edit-format.sh" }
] }
]
}
}
Do not overwrite existing permissions.
```

Matchers are exact and case-sensitive: `Write`, `Edit`, and `Bash`.

Reload with `/clear`, then:

```
Add a comment to the top of server.js saying "// Business Dashboard API Server" but format
it badly with inconsistent spacing.
```

Expected output marker: the hook fires and Prettier normalizes the file.

Exercise 2: Secret Detector - Advisory First (5 min)

Enforcement hooks start advisory: observe, warn, never stop. You promote them only after the false-positive rate has earned it.

```
Create .claude/hooks/pre-write-check.sh that:
- Reads JSON from stdin
- Extracts .tool_input.content
- Exits 0 when empty
- Detects content matching a hardcoded API key, secret, password, token, or sk- credential
- On detection: prints "WARNING: possible hardcoded secret" to stderr and STILL exits 0
(advisory)
- Is executable

Register it in .claude/settings.json as a PreToolUse command hook with matcher "Write".
```

Reload with `/clear`, then:

```
Create test-secret.js containing a clearly fake hardcoded API_KEY value for hook testing.
```

Expected output marker: the warning appears **but the file is still written**. Advisory = exit 0 = the action proceeds. Delete `test-secret.js` before the next exercise.

Exercise 3: Promote It to Blocking (6 min)

Now flip the exit code - and watch what Claude does with the feedback.

```
Edit .claude/hooks/pre-write-check.sh: on detection, print the reason
"Hardcoded secret detected - use an environment variable instead" to stderr
and exit 2 instead of 0. Everything else unchanged.
```

Reload with `/clear`, then rerun the SAME prompt:

```
Create test-secret.js containing a clearly fake hardcoded API_KEY value for hook testing.
```

Expected output marker: the write is **blocked** (exit 2), the reason is fed back to Claude, and - without you asking - Claude typically **reacts to the feedback**: it retries with `process.env.API_KEY` or explains why it cannot proceed. That feedback loop is the point: blocking hooks don't just stop the agent, they steer it.

Confirm the safe path still works:

```
Create test-safe.js containing: const apiKey = process.env.API_KEY
```

Expected output marker: write allowed. Delete both test files afterward.

> *Exit-code semantics (0 = allow, 2 = block) and the structured-JSON decision-output alternative are per the July 2026 hooks reference.*

Exercise 4: Scope the Matcher (3 min)

Your blocker's matcher is `"Write"` - find the gap:

```
Open server.js and use Edit to insert this line near the top: const leaked =
"sk-fake-key-for-testing"
```

Expected output marker: the secret lands - the hook never fired, because `Edit` doesn't match `"Write"`. Remove the line, then close the gap:

```
Change the pre-write-check matcher in .claude/settings.json from "Write" to "Write|Edit",
and make the script also check .tool_input.new_string when .tool_input.content is empty.
```

Reload with `/clear` and retry the Edit prompt. **Expected output marker:** blocked. Lesson: matchers scope rollout - start narrow, widen deliberately, and test the tools you *didn't* match.

Exercise 5: Human-Invoked Skill - /review (4 min)

Custom commands have merged into skills; new workflows use `.claude/skills/<name>/SKILL.md`.

```
Create .claude/skills/review/SKILL.md with:
---
name: review
description: Run a full code review on files explicitly selected by the user.
disable-model-invocation: true
allowed-tools: Read, Grep, Glob, Bash(git diff --name-only)
---

Run a comprehensive code review on $ARGUMENTS using the code-review skill workflow.
Return critical/warning/info totals, the top three fixes, and one verdict:
APPROVE, NEEDS CHANGES, or BLOCK.
If $ARGUMENTS is empty, review files returned by git diff --name-only.
```

Reload and test:

```
/clear
/review src/routes/tasks.js
/review
```

Expected output marker: the first invocation reviews one file; the second follows the changed-files fallback. Claude does not invoke `/review` by itself because `disable-model-invocation` is true.

Exercise 6: Commit the Toolkit (2 min)

```
Stage and commit .claude/hooks/, .claude/skills/, and .claude/settings.json with message
"feat: add enforcement hooks and human-invoked review skill"
```

FINISH MARKER - Success Checklist

- [ ] You can identify the high-value hook event pairs and all 5 handler types
- [ ] PostToolUse formatting runs on Write|Edit
- [ ] Advisory stage: warning printed, write still allowed (exit 0)
- [ ] Blocking stage: write stopped (exit 2) and Claude visibly reacted to the reason
- [ ] Matcher gap found via Edit, closed with Write|Edit, retested
- [ ] `/review <file>` and bare `/review` both work from the skill
- [ ] `disable-model-invocation: true` keeps the workflow human-triggered
- [ ] Toolkit committed

If Stuck

Problem	Recovery
Hook never fires	Check matcher case; run `/clear`; inspect `/doctor`
`jq` unavailable	Ask Claude to implement the hook in Node or Python instead of weakening the check
Hook blocks everything	Pipe safe sample JSON into the script and inspect stdout plus exit status
Claude doesn't react to the block	Make the stderr reason actionable ("use an environment variable instead") - the agent acts on what it can read
`/review` not found	Confirm `.claude/skills/review/SKILL.md`, valid frontmatter, then `/clear`
Prettier unavailable	Install it as a project development dependency

Debrief Questions

1. Which rules require deterministic hooks rather than model judgment?
2. When is advisory the right *endpoint*, not just a stage on the way to blocking?
3. What made Claude self-correct after the block - and what does that imply about writing good deny reasons?
4. Why is invocation authority a better distinction than "command versus skill"?